

The topology of software risk in scientific models

5. Dynamic profiling validation

Arnald Puy

Contents

1 Preliminary	2
2 Helpers and model registry	3
3 Pre-registration	5
4 Building the static risk score from the corrected graphs	6
5 Joining the runtime intensity E_k	7
6 Analysis 1: Path coverage	9
6.1 Unstratified contrast	9
6.2 Within-length deciles (Analysis 1b)	11
7 Analysis 2: Rank correlation between P_k and E_k	13
8 Analysis 3: Reverse predictive check	15
9 Analysis 4: Matched-frequency contrast	17
10 Analysis 5: Dormant high-risk paths	19
11 Cross-model synthesis and decision rule	21
12 Session information	23

1 Preliminary

This document validates the static path-level software-risk framework built in `code_hydrological_models.Rmd` against dynamic profiling. For each model with available runtime traces (ORCHIDEE and PCR-GLOBWB; VIC and HYPE expected) it builds the static risk score P_k from the call graph, joins it to the per-edge runtime call counts, and runs the five analyses pre-registered in `dynamic_validation/pre_registration.md`:

1. **Path coverage.** Are statically high-risk paths preferentially exercised at runtime?
2. **Rank correlation.** Among exercised paths, does P_k order runtime intensity E_k^{sum} correctly?
3. **Reverse predictive check.** Restricting to runtime-heavy paths, does static P_k still discriminate on the underlying node-level metrics?
4. **Matched-frequency contrast.** Does static topology add information beyond execution frequency, when high- and low- P_k paths are matched on E_k^{sum} , length and statement count?
5. **Dormant high-risk paths.** Which statically high-risk paths are not (fully) exercised in the chosen configuration?

Drift-corrected call graphs (2026-07-10). The static call graphs used here are parsed from the *same preprocessed code that actually ran* (ORCHIDEE `build/ppsrc/`; matched PCR-GLOBWB source), supplied in `dynamic_validation/callgraph_csv/`. This removes the codebase-drift between the statically-parsed source and the runtime executable that limited the first pass, and lets P_k and the runtime intensity E_k be computed on one and the same code base. P_k is therefore rebuilt here from the corrected graphs rather than read from the manuscript's original static analysis.

```
# PRELIMINARY FUNCTIONS #####
#####

sensobol::load_packages(c("data.table", "tidyverse", "openxlsx", "scales",
                          "cowplot", "readxl", "ggrepel", "tidytext", "here",
                          "tidygraph", "igraph", "foreach", "parallel", "ggraph",
                          "tools", "purrr", "sensobol", "benchmarkme", "softwareRisk",
                          "boot", "MatchIt", "effsize"))

# Set seed -----

seed <- 123
set.seed(seed)
```

2 Helpers and model registry

The registry lists the drift-corrected static call graphs and their matched runtime traces. ORCHIDEE (model core) is the primary model; ORCHIDEE+IOIPSL (the I/O library included) is a robustness variant that matches the runtime trace more completely at the cost of dragging in library functions without cyclomatic complexity. PCR-GLOBWB is the second model. Append VIC and HYPE here when their graphs arrive.

```
# PATHS #####

repo_root <- getwd()
cg_dir    <- file.path(repo_root, "dynamic_validation", "callgraph_csv", "Callgraphs")
out_dir   <- file.path(repo_root, "dynamic_validation", "results", "per_model")
syn_dir   <- file.path(repo_root, "dynamic_validation", "results", "synthesis")
dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)
dir.create(syn_dir, recursive = TRUE, showWarnings = FALSE)

# MODEL REGISTRY #####
# static  = drift-corrected static call graph (callgraph_edges.csv)
# runtime = matched runtime trace with per-edge runtime_calls
# cc      = model key in datasets/cyclomatic_complexity_functions.csv
# role    = "primary" | "robustness"

model_registry <- data.table(
  model    = c("ORCHIDEE", "ORCHIDEE+IOIPSL", "PCR-GLOBWB"),
  static   = c(file.path(cg_dir, "13_ORCHIDEE-MAN-2025_ORIGINAL", "callgraph_edges.csv"),
               file.path(cg_dir, "13_ORCHIDEE-MAN-2025-AND IOIPSL", "callgraph_edges.csv"),
               file.path(cg_dir, "14_PCR-GLOBWB_model", "callgraph_edges.csv")),
  runtime  = c(file.path(cg_dir, "13_ORCHIDEE-MAN-2025-AND IOIPSL", "runtime_callgraph_edges.csv"),
               file.path(cg_dir, "13_ORCHIDEE-MAN-2025-AND IOIPSL", "runtime_callgraph_edges.csv"),
               file.path(cg_dir, "14_PCR-GLOBWB_model", "runtime_callgraph_edges.csv")),
  cc       = c("ORCHIDEE", "ORCHIDEE", "PCR-GLOBWB"),
  role     = c("primary", "robustness", "primary")
)

available_models <- function() {
  reg <- copy(model_registry)
  reg[, available := file.exists(static) & file.exists(runtime)]
  reg[available == TRUE]
}

# NAME NORMALISATION #####
# Lowercase and strip a leading class qualifier (e.g. BmiPCRGlobWB.set_value ->
# set_value) so static and runtime symbols and the cyclomatic-complexity table
# join on a common key.

norm_fun_name <- function(x) {
  out <- tolower(trimws(as.character(x)))
}
```

```
sub(".*\\.", "", out)
}
```

3 Pre-registration

The decisions in `dynamic_validation/pre_registration.md` were locked before any analysis was run against the dynamic data. Highlights:

- **Path-level aggregation of E_k .** Primary aggregator: `sum` (originally `min`; switched because only a tiny fraction of paths have every edge exercised at least once, making `min` collapse to a near-binary indicator). `min` is reported alongside as a robustness check in Analysis 2.
- **High-risk stratum:** top decile of P_k AND $CV(P_k)$ below the model-specific median CV. **Low-risk stratum:** bottom decile of P_k .
- **Effect-size convention:** common-language A (Vargha & Delaney).
- **Predictive threshold:** $A > 0.55$ at $p < 0.05$ within a comparison; the global decision rule additionally requires Spearman $\rho > 0$ with bootstrap CI strictly above zero, in at least three of the four target models.

```
TOP_DECILE      <- 0.90
BOTTOM_DECILE   <- 0.10
EK_PRIMARY      <- "Ek_sum"    # see deviation log in pre_registration.md
EK_ROBUSTNESS   <- "Ek_min"
N_BOOT         <- 1000L
A_THRESHOLD     <- 0.55
MIN_PATHS_PER_BIN <- 20L
METRICS         <- c("pm_cc", "pm_indeg", "pm_btw") # path-mean C / indeg / btw

# alpha/beta/gamma are the manuscript's node-risk weights.
ALPHA <- 0.6; BETA <- 0.3; GAMMA <- 0.1

color_strata <- c("top_decile" = "#C65D09", "bottom_decile" = "#3B5B92")
```

4 Building the static risk score from the corrected graphs

For each model we build a directed call graph from the drift-corrected edge list, attach cyclomatic complexity (mean-filled where a function is absent from the complexity table), and run `softwareRisk::all_paths_fun` (additive form, $p = 1$) to enumerate entry-to-sink paths and score node and path risk. `softwareRisk::uncertainty_fun` then propagates uncertainty in $(\alpha, \beta, \gamma, p)$ through the path aggregation, giving a distribution of P_k per path from which we take the median $P_{k,q50}$ and the coefficient of variation $CV(P_k)$.

```
# BUILD STATIC P_k #####
#####

build_static <- function(edge_file, cc_model) {
  d <- fread(edge_file)
  el <- unique(data.table(from = norm_fun_name(d$caller),
                          to   = norm_fun_name(d$callee)))
  el <- el[from != "" & to != "" & !is.na(from) & !is.na(to)]

  cc <- fread(file.path(repo_root, "datasets",
                          "cyclomatic_complexity_functions.csv"))[model == cc_model]
  cc[, name_norm := norm_fun_name(name)]
  cc_u <- cc[, .(cyclomatic_complexity = max(cyclomatic_complexity, na.rm = TRUE)),
              by = name_norm]
  mean_cc <- mean(cc_u$cyclomatic_complexity, na.rm = TRUE)

  g <- as_tbl_graph(el, directed = TRUE)
  nm <- igraph::V(as.igraph(g))$name
  ccm <- cc_u[match(nm, name_norm), cyclomatic_complexity]
  ccm[is.na(ccm)] <- mean_cc
  g <- g |> activate(nodes) |> mutate(cyclomatic_complexity = ccm)

  out <- all_paths_fun(graph = g, alpha = ALPHA, beta = BETA, gamma = GAMMA,
                      p = 1, complexity_col = "cyclomatic_complexity")
  ua <- uncertainty_fun(all_paths_out = out, N = 2^10, order = "first")

  paths <- as.data.table(out$paths)
  uap <- as.data.table(ua$paths)
  uap[, P_k_q50 := vapply(uncertainty_analysis, median, numeric(1), na.rm = TRUE)]
  uap[, P_k_mean := vapply(uncertainty_analysis, mean, numeric(1), na.rm = TRUE)]
  uap[, P_k_sd := vapply(uncertainty_analysis, sd, numeric(1), na.rm = TRUE)]
  uap[, CV_k := P_k_sd / pmax(P_k_mean, 1e-12)]
  paths <- merge(paths, uap[, .(path_id, P_k_q50, P_k_mean, P_k_sd, CV_k)],
                by = "path_id")

  list(paths = paths, nodes = as.data.table(out$nodes))
}
```

5 Joining the runtime intensity E_k

`join_runtime` walks each static path, forms its ordered edge list, and looks up the per-edge runtime call counts. A path edge absent from the runtime trace contributes zero calls. It records, per path, whether **any** / **all** of its edges fired, the fraction of edges exercised, and the two aggregators $E_k^{\min}$ and E_k^{sum} . Path-mean cyclomatic complexity, in-degree and betweenness are attached for Analyses 3 and 4.

```
# JOIN RUNTIME E_k #####
#####

load_runtime <- function(runtime_file) {
  rt <- fread(runtime_file)
  rt[, `:=`(cn = norm_fun_name(caller), ce = norm_fun_name(callee))]
  if (!"runtime_calls" %in% names(rt)) rt[, runtime_calls := NA_integer_]
  rt <- rt[, .(runtime_calls = sum(runtime_calls, na.rm = TRUE)), by = .(cn, ce)]
  setkey(rt, cn, ce)
  rt
}

join_runtime <- function(static, runtime_file) {
  paths <- copy(static$paths); nodes <- static$nodes
  rt <- load_runtime(runtime_file)

  res <- paths[, {
    ns <- unlist(path_nodes)
    if (length(ns) < 2L) {
      list(n_edges = 0L, n_ex = 0L, frac_exercised = NA_real_,
           any_exercised = NA, all_exercised = NA,
           Ek_min = NA_real_, Ek_sum = NA_real_)
    } else {
      ed <- data.table(cn = norm_fun_name(ns[-length(ns)]),
                      ce = norm_fun_name(ns[-1L]))
      j <- rt[ed, on = c("cn", "ce")]
      rc <- j$runtime_calls; ex <- !is.na(rc); rc[is.na(rc)] <- 0
      list(n_edges = nrow(ed), n_ex = sum(ex), frac_exercised = sum(ex) / nrow(ed),
           any_exercised = any(ex), all_exercised = all(ex),
           Ek_min = min(rc), Ek_sum = sum(rc))
    }
  }, by = .(path_id)]
  paths <- merge(paths, res, by = "path_id")

  pm <- paths[, {
    ns <- norm_fun_name(unlist(path_nodes))
    sub <- nodes[name %in% ns]
    list(pm_cc = mean(sub$cyclomatic_complexity, na.rm = TRUE),
         pm_indeg = mean(sub$indeg, na.rm = TRUE),
         pm_btw = mean(sub$btw, na.rm = TRUE))
  }]
```

```

}, by = .(path_id)]
merge(paths, pm, by = "path_id")
}

# Build + join for every available model.
reg <- available_models()
joined <- lapply(seq_len(nrow(reg)), function(i) {
  st <- build_static(reg$static[i], reg$cc[i])
  jn <- join_runtime(st, reg$runtime[i])
  jn[, model := reg$model[i]]
  fwrite(jn[, .(model, path_id, hops, P_k_q50, CV_k, n_edges, n_ex,
    frac_exercised, any_exercised, all_exercised, Ek_min, Ek_sum,
    pm_cc, pm_indeg, pm_btw)],
    file.path(out_dir, paste0(gsub("[^A-Za-z0-9]", "_", reg$model[i]),
      "_paths_joined.csv")))
  jn
})
names(joined) <- reg$model

edge_diag <- rbindlist(lapply(seq_len(nrow(reg)), function(i) {
  jn <- joined[[i]]
  data.table(model = reg$model[i], role = reg$role[i],
    paths = nrow(jn), exercised = sum(jn$any_exercised, na.rm = TRUE),
    median_hops = median(jn$hops, na.rm = TRUE))
})))
print(edge_diag)

```

```

##           model           role paths exercised median_hops
##           <char>          <char> <int>      <int>        <num>
## 1:      ORCHIDEE      primary  1459         913           4
## 2: ORCHIDEE+IOIPSL robustness  3214        1886           3
## 3:      PCR-GLOBWB      primary  2239        1412           7

```

6 Analysis 1: Path coverage

6.1 Unstratified contrast

Top vs bottom decile of P_k , three coverage metrics: `frac_any` (at least one edge exercised), `frac_edge` (mean fraction of edges exercised, length-insensitive), and `frac_all` (every edge exercised).

```
# ANALYSIS 1 - PATH COVERAGE #####
#####

coverage_one <- function(dt, model) {
  dt <- dt[!is.na(P_k_q50)]
  qhi <- quantile(dt$P_k_q50, TOP_DECILE, na.rm = TRUE)
  qlo <- quantile(dt$P_k_q50, BOTTOM_DECILE, na.rm = TRUE)
  cvm <- median(dt$CV_k, na.rm = TRUE)
  hi <- dt$P_k_q50 >= qhi & dt$CV_k < cvm
  lo <- dt$P_k_q50 <= qlo
  rbind(
    data.table(model = model, stratum = "top_decile", n = sum(hi),
               frac_any = mean(dt$any_exercised[hi], na.rm = TRUE),
               frac_edge = mean(dt$frac_exercised[hi], na.rm = TRUE),
               frac_all = mean(dt$all_exercised[hi], na.rm = TRUE),
               median_hops = as.numeric(median(dt$hops[hi], na.rm = TRUE))),
    data.table(model = model, stratum = "bottom_decile", n = sum(lo),
               frac_any = mean(dt$any_exercised[lo], na.rm = TRUE),
               frac_edge = mean(dt$frac_exercised[lo], na.rm = TRUE),
               frac_all = mean(dt$all_exercised[lo], na.rm = TRUE),
               median_hops = as.numeric(median(dt$hops[lo], na.rm = TRUE)))
  )
}

coverage_dt <- rbindlist(lapply(reg$model, function(m) coverage_one(joined[[m]], m)))
fwrite(coverage_dt, file.path(syn_dir, "01_coverage_table.csv"))
print(coverage_dt)
```

```
##           model           stratum      n  frac_any frac_edge frac_all
##           <char>           <char> <int>    <num>    <num>    <num>
## 1:      ORCHIDEE      top_decile   145  1.0000000  0.4936946  0.0000000
## 2:      ORCHIDEE bottom_decile   146  0.2123288  0.1666667  0.1232877
## 3: ORCHIDEE+IOIPSL  top_decile   322  0.9968944  0.4907941  0.0000000
## 4: ORCHIDEE+IOIPSL bottom_decile  329  0.2765957  0.2026342  0.1246201
## 5:      PCR-GLOBWB  top_decile   224  1.0000000  0.1439064  0.0000000
## 6:      PCR-GLOBWB bottom_decile  224  0.3214286  0.1568878  0.0625000
## median_hops
##           <num>
## 1:           6.0
## 2:           1.0
## 3:           6.0
```

```
## 4:      2.0
## 5:      8.5
## 6:      2.0
```

```
coverage_long <- melt(coverage_dt,
  id.vars = c("model", "stratum", "n", "median_hops"),
  measure.vars = c("frac_any", "frac_edge", "frac_all"),
  variable.name = "metric", value.name = "value")
coverage_long[, metric := factor(metric,
  levels = c("frac_any", "frac_edge", "frac_all"),
  labels = c("any edge", "mean fraction of edges", "all edges"))]

ggplot(coverage_long, aes(x = model, y = value, fill = stratum)) +
  geom_col(position = position_dodge(0.7), width = 0.6) +
  facet_wrap(~ metric) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1), limits = c(0, 1)) +
  scale_fill_manual(values = color_strata,
    labels = c("bottom decile", "top decile")) +
  labs(x = NULL, y = "coverage", fill = NULL) +
  theme_minimal(base_size = 10) +
  theme(legend.position = "bottom", axis.text.x = element_text(angle = 20, hjust = 1))
```

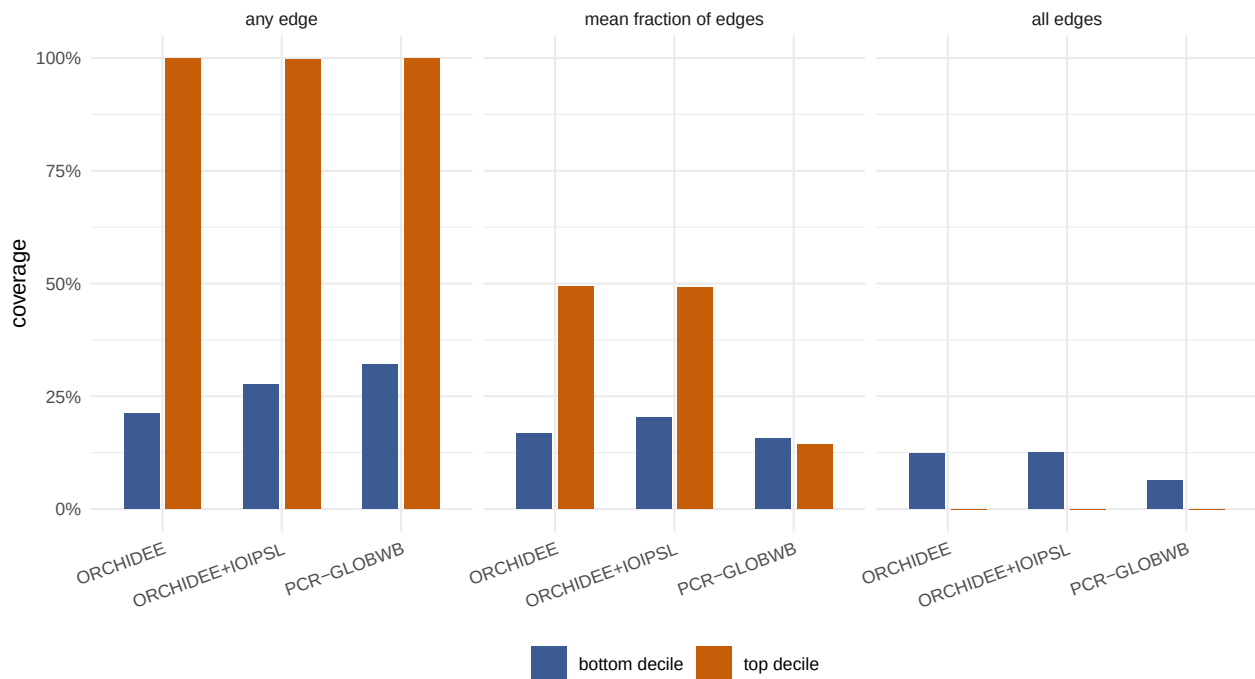


Figure 1: Analysis 1: fraction of paths exercised at runtime by static-risk stratum, three metrics. Top-decile paths are longer than bottom-decile paths (median_hops column), which inflates the any-edge contrast; the length-controlled view is Analysis 1b.

6.2 Within-length deciles (Analysis 1b)

Because P_k is monotone in path length under the saturating-OR aggregator, the unstratified contrast conflates static risk with hops. Here the top vs bottom decile of P_k is taken *within* each hops bin, so length is held constant; the within-bin Wilcoxon test and common-language A summarise discrimination.

```
# ANALYSIS 1b - WITHIN-LENGTH DECILES #####
#####

within_length_one <- function(dt, model) {
  # Plain per-bin split-apply (avoids data.table by-group row recycling).
  x <- data.table(hops = dt$hops, P_k_q50 = dt$P_k_q50,
                  frac_exercised = dt$frac_exercised[!is.na(P_k_q50)])
  x[, hb := cut(hops, c(0, 1, 2, 4, 7, Inf), labels = c("1", "2", "3-4", "5-7", "8+"))]
  rbindlist(lapply(levels(x$hb), function(b) {
    sub <- x[hb == b]
    if (nrow(sub) < MIN_PATHS_PER_BIN || length(unique(sub$P_k_q50)) < 2) return(NULL)
    q1 <- quantile(sub$P_k_q50, TOP_DECILE); q0 <- quantile(sub$P_k_q50, BOTTOM_DECILE)
    xt <- sub$frac_exercised[sub$P_k_q50 >= q1]; xb <- sub$frac_exercised[sub$P_k_q50 <= q0]
    if (length(xt) < 3 || length(xb) < 3) return(NULL)
    data.table(model = model, hb = b, n_top = length(xt), n_bot = length(xb),
               med_top = median(xt, na.rm = TRUE), med_bot = median(xb, na.rm = TRUE),
               wilcox_p = suppressWarnings(wilcox.test(xt, xb)$p.value),
               A = tryCatch(effsize::VD.A(xt, xb)$estimate, error = function(e) NA_real_))
  })))
}

within_dt <- rbindlist(lapply(reg$model, function(m) within_length_one(joined[[m]], m)),
                       fill = TRUE)
fwrite(within_dt, file.path(syn_dir, "01b_within_length.csv"))
print(within_dt)
```

##	model	hb	n_top	n_bot	med_top	med_bot	wilcox_p
##	<char>	<char>	<int>	<int>	<num>	<num>	<num>
## 1:	ORCHIDEE	1	12	12	0.0000000	1.0000000	1.911835e-06
## 2:	ORCHIDEE	2	33	31	0.0000000	0.0000000	8.821716e-03
## 3:	ORCHIDEE	3-4	47	47	0.5000000	0.0000000	6.296432e-09
## 4:	ORCHIDEE	5-7	58	59	0.5000000	0.0000000	9.256026e-13
## 5:	ORCHIDEE+IOIPSL	1	45	23	0.0000000	1.0000000	4.066674e-11
## 6:	ORCHIDEE+IOIPSL	2	80	77	0.0000000	0.0000000	4.057258e-01
## 7:	ORCHIDEE+IOIPSL	3-4	114	115	0.2500000	0.0000000	4.125006e-10
## 8:	ORCHIDEE+IOIPSL	5-7	103	103	0.5000000	0.2000000	4.423861e-16
## 9:	ORCHIDEE+IOIPSL	8+	8	8	0.6250000	0.3750000	6.160025e-03
## 10:	PCR-GLOBWB	1	7	7	0.0000000	0.0000000	NaN
## 11:	PCR-GLOBWB	2	12	12	0.0000000	0.0000000	1.658240e-01
## 12:	PCR-GLOBWB	3-4	44	45	0.0000000	0.2500000	6.131598e-04
## 13:	PCR-GLOBWB	5-7	84	84	0.1428571	0.1666667	8.211308e-01
## 14:	PCR-GLOBWB	8+	79	79	0.1111111	0.1250000	3.755189e-01

```
##           A
##      <num>
##  1: 0.0000000
##  2: 0.4032258
##  3: 0.8338615
##  4: 0.8749269
##  5: 0.1304348
##  6: 0.4685065
##  7: 0.7300534
##  8: 0.8251485
##  9: 0.9062500
## 10: 0.5000000
## 11: 0.4166667
## 12: 0.3128788
## 13: 0.5099206
## 14: 0.4595417

wl_plot <- within_dt[!is.na(A)]
if (nrow(wl_plot)) {
  ggplot(wl_plot, aes(x = hb, y = A, fill = model)) +
    geom_col(position = position_dodge(0.7), width = 0.6) +
    geom_hline(yintercept = A_THRESHOLD, linetype = "dashed") +
    geom_hline(yintercept = 0.5, linetype = "dotted", colour = "grey60") +
    coord_cartesian(ylim = c(0, 1)) +
    labs(x = "path length (hops)", y = "effect size A (top vs bottom decile)",
         fill = NULL,
         caption = "dashed = A 0.55 threshold; dotted = A 0.5 (no discrimination)") +
    theme_minimal(base_size = 10) + theme(legend.position = "bottom")
}
```

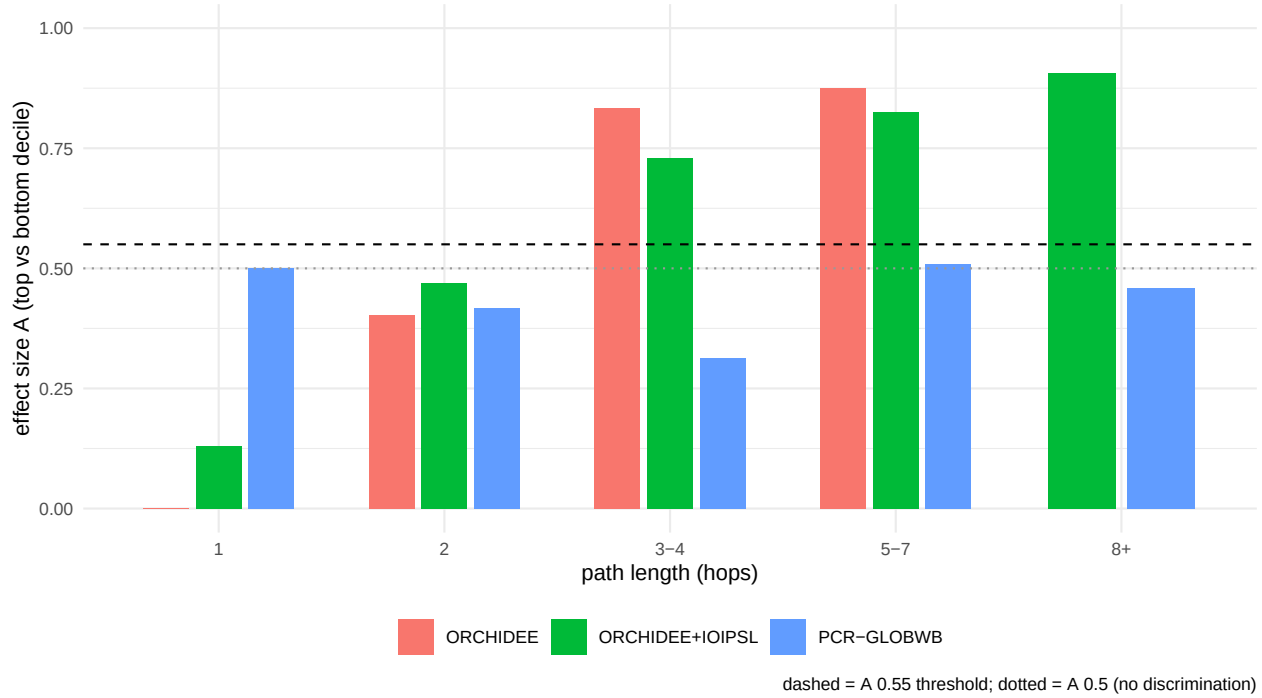


Figure 2: Analysis 1b: mean fraction of edges exercised, top vs bottom decile of P_k within each hops bin. Decile cuts are taken within-bin so length is controlled by construction.

7 Analysis 2: Rank correlation between P_k and E_k

Among paths that runtime actually exercised, does P_k rank-order runtime intensity? Spearman ρ on E_k^{sum} (primary) with a percentile bootstrap CI, plus E_k^{min} as a robustness check.

```
# ANALYSIS 2 - RANK CORRELATION #####
#####

rank_corr_one <- function(dt, model) {
  ex <- dt[any_exercised == TRUE & !is.na(get(EK_PRIMARY)) & !is.na(P_k_q50)]
  if (nrow(ex) < 10) return(NULL)
  rho <- suppressWarnings(cor(ex$P_k_q50, ex[[EK_PRIMARY]], method = "spearman"))
  tau <- suppressWarnings(cor(ex$P_k_q50, ex[[EK_PRIMARY]], method = "kendall"))
  rmin <- suppressWarnings(cor(ex$P_k_q50, ex[[EK_ROBUSTNESS]], method = "spearman"))
  bo <- boot::boot(ex, function(d, i)
    suppressWarnings(cor(d$P_k_q50[i], d[[EK_PRIMARY]][i], method = "spearman")),
    R = N_BOOT)
  ci <- tryCatch(boot::boot.ci(bo, type = "perc")$percent[1, c(4, 5)],
    error = function(e) c(NA_real_, NA_real_))
  data.table(model = model, n_exercised = nrow(ex),
    spearman_sum = rho, ci_lo = ci[1], ci_hi = ci[2],
    kendall_sum = tau, spearman_min = rmin)
}

corr_dt <- rbindlist(lapply(reg$model, function(m) rank_corr_one(joined[[m]], m)),
```

```

      fill = TRUE)
fwrite(corr_dt, file.path(syn_dir, "02_rank_correlation.csv"))
print(corr_dt)

##           model n_exercised spearman_sum      ci_lo      ci_hi kendall_sum
##           <char>         <int>         <num>      <num>      <num>      <num>
## 1:      ORCHIDEE           913      0.4971958 0.4406202 0.5473802  0.3679732
## 2: ORCHIDEE+IOIPSL          1886      0.2433461 0.1939382 0.2892486  0.1713528
## 3:      PCR-GLOBWB          1412      0.2390355 0.1839138 0.2892955  0.1726507
##      spearman_min
##           <num>
## 1:      -0.2725521
## 2:      -0.3108496
## 3:      -0.1803274

scatter_panels <- lapply(reg$model, function(m) {
  ex <- joined[[m]][any_exercised == TRUE & !is.na(get(EK_PRIMARY))]
  if (!nrow(ex)) return(NULL)
  r <- corr_dt[model == m]
  ggplot(ex, aes(x = P_k_q50, y = log10(get(EK_PRIMARY) + 1))) +
    geom_point(alpha = 0.3, size = 0.7) +
    geom_smooth(method = "loess", se = FALSE, colour = "#C65D09") +
    labs(title = sprintf("%s (rho=%.2f [%.2f,%.2f], n=%d)",
                        m, r$spearman_sum, r$ci_lo, r$ci_hi, r$n_exercised),
         x = expression(P[k]), y = expression(log[10]*("E[k]^{sum}+1*")")) +
    theme_minimal(base_size = 9)
})

scatter_panels <- scatter_panels[!vapply(scatter_panels, is.null, logical(1))]
if (length(scatter_panels)) cowplot::plot_grid(plotlist = scatter_panels, nrow = 1)

## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'

```

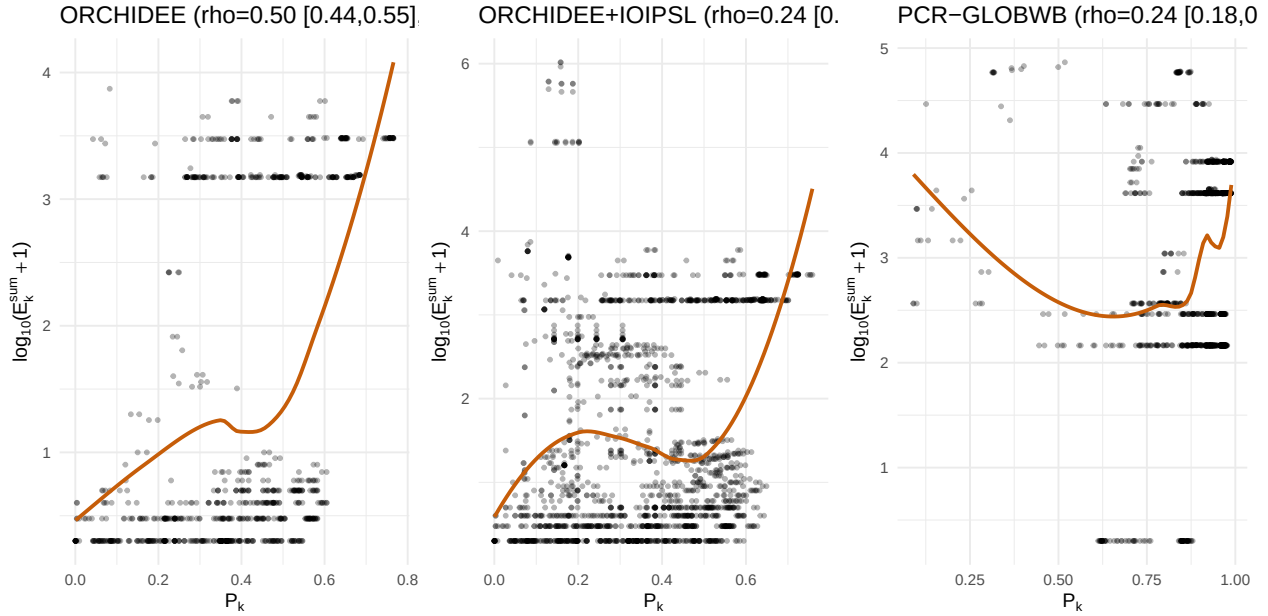


Figure 3: Analysis 2: P_k against $\log_{10}(E_k^{\text{sum}} + 1)$ for exercised paths, per model; Spearman rho annotated.

8 Analysis 3: Reverse predictive check

Restrict to the top decile of paths by E_k^{sum} (runtime-heavy), then within that subset split at the median P_k and compare path-mean cyclomatic complexity, in-degree and betweenness. *Holding execution frequency fixed, does static topology still discriminate?*

ANALYSIS 3 - REVERSE PREDICTIVE CHECK

```
reverse_one <- function(dt, model) {
  ex <- dt[any_exercised == TRUE & !is.na(get(EK_PRIMARY))]
  if (nrow(ex) < 16) return(NULL)
  qr <- quantile(ex[[EK_PRIMARY]], TOP_DECILE, na.rm = TRUE)
  hot <- ex[get(EK_PRIMARY) >= qr]
  if (nrow(hot) < 8) return(NULL)
  qp <- quantile(hot$P_k_q50, 0.5)
  hot[, st := ifelse(P_k_q50 >= qp, "hi", "lo")]
  rbindlist(lapply(METRICS, function(mt) {
    a <- hot[st == "hi", get(mt)]; b <- hot[st == "lo", get(mt)]
    data.table(model = model, metric = mt, n_hi = length(a), n_lo = length(b),
               median_hi = median(a, na.rm = TRUE), median_lo = median(b, na.rm = TRUE),
               wilcox_p = suppressWarnings(wilcox.test(a, b)$p.value),
               A = tryCatch(efsize::VD.A(a, b)$estimate, error = function(e) NA_real_))
  })))
}

reverse_dt <- rbindlist(lapply(reg$model, function(m) reverse_one(joined[[m]], m)),
                        fill = TRUE)
```

```
reverse_dt[, metric := factor(metric, levels = METRICS,
                              labels = c("cyclomatic", "in-degree", "betweenness"))]
fwrite(reverse_dt, file.path(syn_dir, "03_reverse_predictive.csv"))
print(reverse_dt)
```

	model	metric	n_hi	n_lo	median_hi	median_lo	wilcox_p
	<char>	<fctr>	<int>	<int>	<num>	<num>	<num>
## 1:	ORCHIDEE	cyclomatic	53	53	82.000000	45.68286	8.824840e-10
## 2:	ORCHIDEE	in-degree	53	53	1.333333	1.20000	5.232504e-03
## 3:	ORCHIDEE	betweenness	53	53	110.777056	74.93939	2.268785e-12
## 4:	ORCHIDEE+IOIPSL	cyclomatic	97	97	82.121907	16.73144	4.089479e-29
## 5:	ORCHIDEE+IOIPSL	in-degree	97	97	1.400000	5.80000	6.422899e-01
## 6:	ORCHIDEE+IOIPSL	betweenness	97	97	114.348485	48.66667	4.367906e-32
## 7:	PCR-GLOBWB	cyclomatic	125	124	29.000000	21.43077	1.080716e-08
## 8:	PCR-GLOBWB	in-degree	125	124	6.111111	5.50000	3.748483e-07
## 9:	PCR-GLOBWB	betweenness	125	124	208.281818	157.73452	4.529270e-20
##	A						
##	<num>						
## 1:	0.8454966						
## 2:	0.6559274						
## 3:	0.8915984						
## 4:	0.9623764						
## 5:	0.4807100						
## 6:	0.9891062						
## 7:	0.7096452						
## 8:	0.6861613						
## 9:	0.8363226						

9 Analysis 4: Matched-frequency contrast

The cleanest test of “does static topology add information beyond execution frequency”: match each top-decile high- P_k path to a bottom-decile low- P_k path with comparable E_k^{sum} , hops and statement count (Mahalanobis nearest-neighbour with replacement), then compare static metrics on the matched set.

```
# ANALYSIS 4 - MATCHED-FREQUENCY CONTRAST #####
#####

matched_one <- function(dt, model) {
  dt <- dt[!is.na(P_k_q50)]
  qhi <- quantile(dt$P_k_q50, TOP_DECILE, na.rm = TRUE)
  qlo <- quantile(dt$P_k_q50, BOTTOM_DECILE, na.rm = TRUE)
  cvm <- median(dt$CV_k, na.rm = TRUE)
  dm <- dt[any_exercised == TRUE & !is.na(get(EK_PRIMARY))]
  dm[, grp := fifelse(P_k_q50 >= qhi & CV_k < cvm, 1L,
                     fifelse(P_k_q50 <= qlo,          0L, NA_integer_))]
  dm <- dm[!is.na(grp)]
  if (sum(dm$grp == 1) < 5 || sum(dm$grp == 0) < 5) return(NULL)
  dm[, stmt := pm_cc * hops]
  mo <- tryCatch(MatchIt::matchit(grp ~ Ek_sum + hops + stmt, data = dm,
                                method = "nearest", distance = "mahalanobis", replace = TRUE),
                error = function(e) NULL)
  if (is.null(mo)) return(NULL)
  md <- MatchIt::match.data(mo); np <- sum(md$grp == 1)
  rbindlist(lapply(METRICS, function(mt) {
    a <- md[[mt]][md$grp == 1]; b <- md[[mt]][md$grp == 0]
    data.table(model = model, metric = mt, n_pairs = np,
               median_hi = median(a, na.rm = TRUE), median_lo = median(b, na.rm = TRUE),
               wilcox_p = suppressWarnings(wilcox.test(a, b)$p.value),
               A = tryCatch(efsize::VD.A(a, b)$estimate, error = function(e) NA_real_),
               exploratory = np < 15L)
  })))
}

matched_dt <- rbindlist(lapply(reg$model, function(m) matched_one(joined[[m]], m)),
                       fill = TRUE)
matched_dt[, metric := factor(metric, levels = METRICS,
                              labels = c("cyclomatic", "in-degree", "betweenness"))]
fwrite(matched_dt, file.path(syn_dir, "04_matched_contrast.csv"))
print(matched_dt)
```

##	model	metric	n_pairs	median_hi	median_lo	wilcox_p
##	<char>	<fctr>	<int>	<num>	<num>	<num>
## 1:	ORCHIDEE	cyclomatic	145	64.71643	27.000000	0.0044543961
## 2:	ORCHIDEE	in-degree	145	16.50000	2.000000	0.0853801765
## 3:	ORCHIDEE	betweenness	145	86.07323	1.000000	0.0030275603
## 4:	ORCHIDEE+IOIPSL	cyclomatic	321	48.96643	21.844432	0.0355790729

## 5:	ORCHIDEE+IOIPSL	in-degree	321	16.37500	2.175000	0.0067163268
## 6:	ORCHIDEE+IOIPSL	betweenness	321	82.44129	4.033333	0.0005887191
## 7:	PCR-GLOBWB	cyclomatic	224	30.21111	20.839286	0.0565577505
## 8:	PCR-GLOBWB	in-degree	224	5.90000	2.857143	0.0159064857
## 9:	PCR-GLOBWB	betweenness	224	240.45556	7.191964	0.0141592346
##	A exploratory					
##	<num>	<lgcl>				
## 1:	0.9816092	FALSE				
## 2:	0.7908046	FALSE				
## 3:	1.0000000	FALSE				
## 4:	0.8060748	FALSE				
## 5:	0.8944704	FALSE				
## 6:	1.0000000	FALSE				
## 7:	0.8928571	FALSE				
## 8:	0.9955357	FALSE				
## 9:	1.0000000	FALSE				

10 Analysis 5: Dormant high-risk paths

Of the within-length top-decile high-risk paths in each length bin, how many were not (fully) exercised? *Loose* dormancy = no edge fired; *strict* dormancy = at least one edge did not fire.

```
# ANALYSIS 5 - DORMANT HIGH-RISK PATHS #####
#####

dormant_one <- function(dt, model) {
  x <- data.table(hops = dt$hops, P_k_q50 = dt$P_k_q50, CV_k = dt$CV_k,
                 any_exercised = dt$any_exercised,
                 all_exercised = dt$all_exercised)[!is.na(P_k_q50)]
  x[, hb := cut(hops, c(0, 1, 2, 4, 7, Inf), labels = c("1", "2", "3-4", "5-7", "8+"))]
  rbindlist(lapply(levels(x$hb), function(b) {
    sub <- x[hb == b]
    if (nrow(sub) < MIN_PATHS_PER_BIN || length(unique(sub$P_k_q50)) < 2) return(NULL)
    q1 <- quantile(sub$P_k_q50, TOP_DECILE); cm <- median(sub$CV_k, na.rm = TRUE)
    h <- sub[P_k_q50 >= q1 & CV_k < cm]
    data.table(model = model, hb = b, n_top = nrow(h),
              dormant_loose = sum(h$any_exercised == FALSE | is.na(h$any_exercised)),
              dormant_strict = sum(h$all_exercised == FALSE | is.na(h$all_exercised)))
  })))
}

dormant_dt <- rbindlist(lapply(reg$model, function(m) dormant_one(joined[[m]], m)),
                       fill = TRUE)
dormant_dt[, `:=`(frac_loose = dormant_loose / n_top,
                  frac_strict = dormant_strict / n_top)]
fwrite(dormant_dt, file.path(syn_dir, "05_dormancy.csv"))
print(dormant_dt)
```

##	model	hb	n_top	dormant_loose	dormant_strict	frac_loose
##	<char>	<char>	<int>	<int>	<int>	<num>
## 1:	ORCHIDEE	1	3	3	3	1.00000000
## 2:	ORCHIDEE	2	29	29	29	1.00000000
## 3:	ORCHIDEE	3-4	43	1	43	0.02325581
## 4:	ORCHIDEE	5-7	44	0	44	0.00000000
## 5:	ORCHIDEE+IOIPSL	1	3	3	3	1.00000000
## 6:	ORCHIDEE+IOIPSL	2	17	3	9	0.17647059
## 7:	ORCHIDEE+IOIPSL	3-4	101	10	101	0.09900990
## 8:	ORCHIDEE+IOIPSL	5-7	74	0	74	0.00000000
## 9:	ORCHIDEE+IOIPSL	8+	8	0	8	0.00000000
## 10:	PCR-GLOBWB	1	7	7	7	1.00000000
## 11:	PCR-GLOBWB	2	6	6	6	1.00000000
## 12:	PCR-GLOBWB	3-4	44	32	44	0.72727273
## 13:	PCR-GLOBWB	5-7	84	1	84	0.01190476
## 14:	PCR-GLOBWB	8+	79	0	79	0.00000000
##	frac_strict					
##	<num>					

```
## 1: 1.0000000
## 2: 1.0000000
## 3: 1.0000000
## 4: 1.0000000
## 5: 1.0000000
## 6: 0.5294118
## 7: 1.0000000
## 8: 1.0000000
## 9: 1.0000000
## 10: 1.0000000
## 11: 1.0000000
## 12: 1.0000000
## 13: 1.0000000
## 14: 1.0000000
```

11 Cross-model synthesis and decision rule

```
# CROSS-MODEL SYNTHESIS #####
#####

# Headline effect sizes on cyclomatic complexity from the two harder tests.
rev_cc <- reverse_dt[metric == "cyclomatic", .(model, A_reverse_cc = A)]
mat_cc <- matched_dt[metric == "cyclomatic", .(model, A_matched_cc = A, n_pairs)]
mat_btw <- matched_dt[metric == "betweenness", .(model, A_matched_btw = A)]

headline <- Reduce(function(a, b) merge(a, b, by = "model", all = TRUE),
  list(corr_dt[, .(model, spearman_sum, ci_lo, ci_hi, n_exercised)],
    rev_cc, mat_cc, mat_btw))
fwrite(headline, file.path(syn_dir, "06_cross_model_headline.csv"))
print(headline)

## Key: <model>
##           model spearman_sum      ci_lo      ci_hi n_exercised A_reverse_cc
##           <char>      <num>      <num>      <num>      <int>      <num>
## 1:      ORCHIDEE      0.4971958 0.4406202 0.5473802        913      0.8454966
## 2: ORCHIDEE+IOIPSL      0.2433461 0.1939382 0.2892486       1886      0.9623764
## 3:      PCR-GLOBWB      0.2390355 0.1839138 0.2892955       1412      0.7096452
##   A_matched_cc n_pairs A_matched_btw
##           <num>  <int>      <num>
## 1:    0.9816092   145          1
## 2:    0.8060748   321          1
## 3:    0.8928571   224          1

# Pre-registered decision rule: Spearman rho > 0 with CI_lo > 0, AND A > 0.55
# in at least one of the harder tests (reverse predictive or matched contrast).
decision <- headline[, .(model,
  spearman_pos = spearman_sum > 0 & ci_lo > 0,
  harder_A_pass = (A_reverse_cc > A_THRESHOLD) | (A_matched_cc > A_THRESHOLD))]
decision[, passes := spearman_pos & harder_A_pass]
fwrite(decision, file.path(syn_dir, "06_decision_rule.csv"))
print(decision)

## Key: <model>
##           model spearman_pos harder_A_pass passes
##           <char>      <lgcl>      <lgcl> <lgcl>
## 1:      ORCHIDEE      TRUE      TRUE      TRUE
## 2: ORCHIDEE+IOIPSL      TRUE      TRUE      TRUE
## 3:      PCR-GLOBWB      TRUE      TRUE      TRUE

n_primary <- reg[role == "primary", model]
cat(sprintf("\n[decision rule] %d of %d primary models pass both legs (%s).\n",
  sum(decision[model %in% n_primary, passes], na.rm = TRUE),
  length(n_primary), paste(n_primary, collapse = ", ")))
```

```
##
## [decision rule] 2 of 2 primary models pass both legs (ORCHIDEE, PCR-GLOBWB).
cat("Full pre-registered claim requires >= 3 of 4 target models (VIC + HYPE pending).\n")

## Full pre-registered claim requires >= 3 of 4 target models (VIC + HYPE pending).

p_rho <- ggplot(corr_dt, aes(x = model, y = spearman_sum)) +
  geom_pointrange(aes(ymin = ci_lo, ymax = ci_hi)) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(title = expression("Spearman " ~ rho ~ "(P"[k]*", E"[k]^{"sum}*")"),
       x = NULL, y = NULL) +
  theme_minimal(base_size = 9) + theme(axis.text.x = element_text(angle = 20, hjust = 1))

effect_dt <- rbind(
  reverse_dt[metric == "cyclomatic", .(model, test = "reverse predictive", A)],
  matched_dt[metric == "cyclomatic", .(model, test = "matched contrast", A)])
p_A <- ggplot(effect_dt, aes(x = model, y = A, fill = test)) +
  geom_col(position = position_dodge(0.7), width = 0.6) +
  geom_hline(yintercept = A_THRESHOLD, linetype = "dashed") +
  coord_cartesian(ylim = c(0, 1)) +
  labs(title = "Effect size A on cyclomatic complexity", x = NULL, y = NULL, fill = NULL) +
  theme_minimal(base_size = 9) +
  theme(legend.position = "bottom", axis.text.x = element_text(angle = 20, hjust = 1))

cowplot::plot_grid(p_rho, p_A, ncol = 2, rel_widths = c(0.45, 0.55))
```

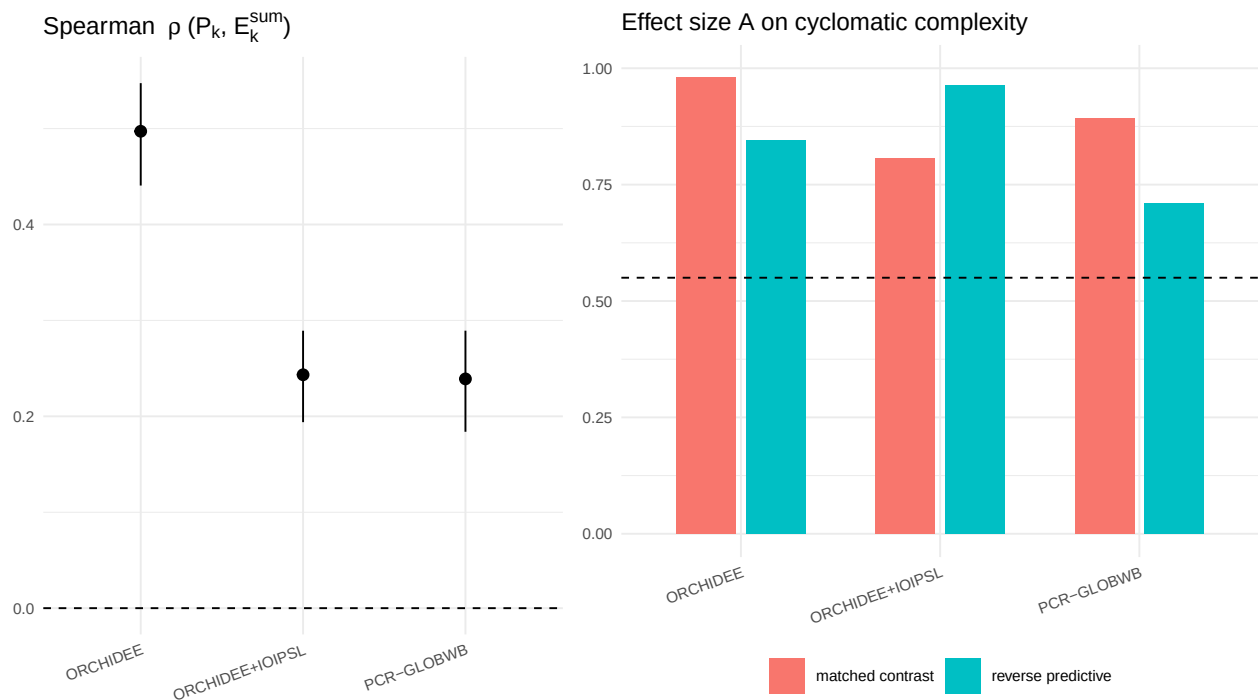


Figure 4: Cross-model synthesis: Spearman $\rho(P_k, E_k^{\text{sum}})$ with bootstrap CI, and effect size A on cyclomatic complexity from the reverse-predictive and matched-frequency tests. Dashed line at $A = 0.55$.

12 Session information

```
# SESSION INFORMATION #####  
#####
```

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)  
## Platform: aarch64-apple-darwin20  
## Running under: macOS Tahoe 26.5.1  
##  
## Matrix products: default  
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/Libraries/libBLAS.dylib  
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  
##  
## locale:  
## [1] C  
##  
## time zone: Europe/London  
## tzcode source: internal  
##  
## attached base packages:  
## [1] tools parallel stats graphics grDevices utils datasets  
## [8] methods base  
##  
## other attached packages:  
## [1] effsize_0.8.1 MatchIt_4.7.2 boot_1.3-32 softwareRisk_0.2.0  
## [5] benchmarkme_1.0.8 sensobol_1.2.0 ggraph_2.2.2 foreach_1.5.2  
## [9] igraph_2.2.1 tidygraph_1.3.1 here_1.0.2 tidytext_0.4.3  
## [13] ggrepel_0.9.6 readxl_1.4.5 cowplot_1.2.0.9000 scales_1.4.0  
## [17] openxlsx_4.2.8 lubridate_1.9.4 forcats_1.0.1 stringr_1.6.0  
## [21] dplyr_1.1.4 purrr_1.2.1 readr_2.2.0 tidyr_1.3.2  
## [25] tibble_3.3.1 ggplot2_4.0.3.9000 tidyverse_2.0.0 data.table_1.18.4  
##  
## loaded via a namespace (and not attached):  
## [1] tidyselect_1.2.1 viridisLite_0.4.3 farver_2.1.2  
## [4] viridis_0.6.5 S7_0.2.2 fastmap_1.2.0  
## [7] tweenr_2.0.3 janeaustenr_1.0.0 digest_0.6.39  
## [10] timechange_0.3.0 lifecycle_1.0.5 tokenizers_0.3.0  
## [13] magrittr_2.0.5 compiler_4.5.2 rlang_1.2.0  
## [16] yaml_2.3.12 knitr_1.51 graphlayouts_1.2.2  
## [19] RColorBrewer_1.1-3 withr_3.0.3 grid_4.5.2  
## [22] polyclip_1.10-7 iterators_1.0.14 MASS_7.3-65  
## [25] cli_3.6.6 rmarkdown_2.30 generics_0.1.4  
## [28] otl_0.2.0 httr_1.4.8 tzdb_0.5.0  
## [31] cachem_1.1.0 ggforce_0.5.0 cellranger_1.1.0  
## [34] vctrs_0.7.3 Matrix_1.7-4 hms_1.1.4  
## [37] ineq_0.2-13 glue_1.8.1 benchmarkmeData_1.0.4
```

```
## [40] codetools_0.2-20      stringi_1.8.7          gtable_0.3.6
## [43] pillar_1.11.1         htmltools_0.5.9        R6_2.6.1
## [46] Rdpack_2.6.6          doParallel_1.0.17      rprojroot_2.1.1
## [49] evaluate_1.0.5        lattice_0.22-7         backports_1.5.0
## [52] rbibutils_2.4.1       SnowballC_0.7.1        memoise_2.0.1
## [55] Rcpp_1.1.1-1.1        zip_2.3.3              gridExtra_2.3
## [58] xfun_0.55             pkgconfig_2.0.3
```

```
cat("Machine:      "); print(get_cpu()$model_name)
```

```
## Machine:
```

```
## [1] "Apple M1 Max"
```

```
cat("Num cores:    "); print(detectCores(logical = FALSE))
```

```
## Num cores:
```

```
## [1] 10
```

```
cat("Num threads: "); print(detectCores(logical = TRUE))
```

```
## Num threads:
```

```
## [1] 10
```